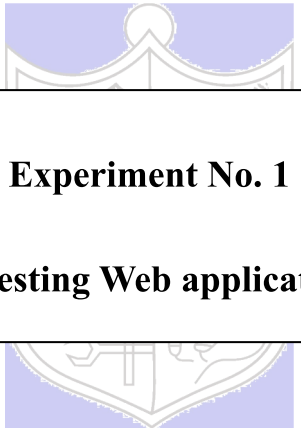


Experiment No. 1

Title: Mini Project on testing Web application/Mobile application

Batch: B–1 **Roll No.: 16010422234** **Experiment No.: 1**

Aim: To develop a Web application/Mobile application and test it by using open source software testing tools.

Resources needed: Internet, Open source testing tool, Application development software

---

Theory:

In software testing, test automation is the use of special software (separate from the software being tested) to control the execution of tests and the comparison of actual outcomes with predicted outcomes. Test automation can automate some repetitive but necessary tasks in a formalized testing process already in place, or add additional testing that would be difficult to perform manually.

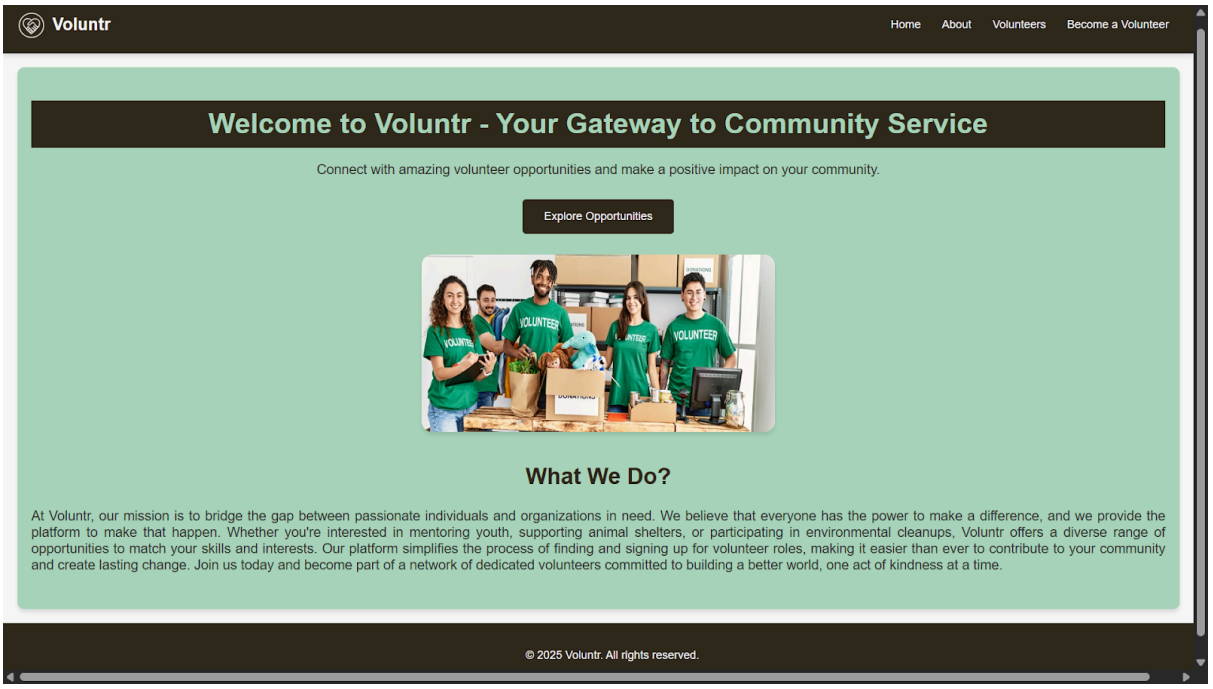
Some software testing tasks, such as extensive low-level interface regression testing, can be laborious and time consuming to do manually. In addition, a manual approach might not always be effective in finding certain classes of defects. Test automation offers a possibility to perform these types of testing effectively. Once automated tests have been developed, they can be run quickly and repeatedly. Many times, this can be a cost-effective method for regression testing of software products that have a long maintenance life. Even minor patches over the lifetime of the application can cause existing features to break which were working at an earlier point in time.

Activities:

1. Develop a sample web/mobile application.
2. Design software testing methodology in detail for the selected application.
3. Explore open source software testing tools for testing web/mobile applications.
4. Document the functionality of testing tools with its features.
5. Document sample application features.
6. Create test cases and test scripts.
7. Run and document test cases.

Results: (Document printout as per the format)

1. Develop a sample web/mobile application.



2. Design software testing methodology in detail for the selected application.

Section	What I did?	Key files / configs	Commands	Outcome
Testing Methodology	Defined pyramid: Unit/UI → Accessibility → E2E → Perf/SEO.	src/setupTests.js, playwright.config.ts, .lighthouserc.json	—	Methodology documented

Unit & Accessibility Testing (Jest/RTL + axe)

src__tests__\Navbar.test.jsx

What it does: This test checks your <Navbar /> component.

Explanation: It uses render to load the component into a virtual test environment. Then, it uses screen.getByRole('link', ...) to find the navigation links. The expect(...).toBeInTheDocument() line is an assertion that confirms links with text like "home" , "volunteers" , and "become a volunteer" are all successfully rendered.

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import Navbar from '../components/Navbar';

jest.mock(
```

```
  'react-router-dom',
  () => ({
    Link: ({ to, children }) => <a href={to}>{children}</a>,
  }),
  { virtual: true }
);

test('renders key nav links', () => {
  render(<Navbar />);
           expect(screen.getByRole('link',      {      name:      /home/i
})).toBeInTheDocument();
         expect(screen.getByRole('link',     {     name:     /volunteers/i
})).toBeInTheDocument();
    expect(screen.getByRole('link',   {   name:   /become   a   volunteer/i
})).toBeInTheDocument();
});
```

src__tests__\HomePage.test.jsx

What it does: This test verifies that the main heading on the home page appears.

Explanation: It renders the <HomePage /> component. It then looks for an element with the "heading" role that contains the text "welcome to voluntr". This confirms the main welcome message is on the page.

```
import { render, screen } from '@testing-library/react';
import HomePage from '../pages/HomePage';

test('shows welcome heading', () => {
  render(<HomePage />);
    expect(screen.getByRole('heading',  {  name:  /welcome  to  voluntr/i
})).toBeInTheDocument();
});
```

src__tests__\VolunteerFormPage.test.jsx

What it does: It checks the form's validation logic.

Explanation:
1. It renders the <VolunteerFormPage />.
2. It finds the "submit" button and asserts expect(submit).toBeDisabled(), proving the form is invalid by default.

3. It uses userEvent.type to simulate a user typing invalid data (e.g., a short skill description). It checks that the button is still disabled.
4. It then simulates clearing the fields and typing valid data (a proper email and a longer skill description) .
5. It asserts expect(submit).toBeEnabled(), proving the validation logic worked.
6. Finally, it simulates clicking the enabled button and expects to find the "thank you!" success message.

```
import { render, screen } from '@testing-library/react';
import { act } from 'react';
import userEvent from '@testing-library/user-event';
import VolunteerFormPage from '../pages/VolunteerFormPage';

test('submit disabled until valid; shows success on submit', async ()
=> {
  render(<VolunteerFormPage />);

  const name   = screen.getByLabelText(/name/i);
  const email  = screen.getByLabelText(/email/i);
  const skills = screen.getByLabelText(/your skills/i);
  const submit = screen.getByRole('button', { name: /submit/i });

  expect(submit).toBeDisabled();

  await act(async () => {
    await userEvent.type(name, 'Ada Lovelace');
    await userEvent.type(email, 'not-an-email');
    await userEvent.type(skills, 'short');
  });
  expect(submit).toBeDisabled();

  await act(async () => {
    await userEvent.clear(email);
    await userEvent.type(email, 'ada@example.com');
    await userEvent.clear(skills);
       await userEvent.type(skills, 'This description is longer than
twenty characters.');
  });
  expect(submit).toBeEnabled();

  await act(async () => {
    await userEvent.click(submit);
  });
```

```
  expect(await screen.findByText(/thank you!/i)).toBeInTheDocument();
});
```

src__tests__\HomePage.a11y.test.jsx

What it does: This is an accessibility test.

Explanation: It uses the axe tool from jest-axe. It renders the <HomePage /> , then runs axe on the component's HTML. The assertion expect(results.violations).toEqual([]) checks that axe found zero accessibility violations, proving the page is accessible.

```
import { render } from '@testing-library/react';
import { axe, toHaveNoViolations } from 'jest-axe';
import HomePage from '../pages/HomePage';
expect.extend(toHaveNoViolations);

test('home page has no basic a11y violations', async () => {
  const { container } = render(<HomePage />);
  const results = await axe(container);
  expect(results.violations).toEqual([]);
});
```


End-to-End Testing (Playwright)

These tests check user "flows" from start to finish in a real browser environment.

playwright.config.ts

What it does: This is the configuration file for Playwright.

Explanation: It tells Playwright where to find the tests (testDir: './tests') , what the application's URL is (baseURL: 'http://localhost:3000') , and which browsers to run the tests in (chromium, firefox, webkit) . It also tells Playwright how to start your server (webServer: { command: 'npm start', ... }).

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  use: { baseURL: 'http://localhost:3000' },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
```

```
    { name: 'webkit',   use: { ...devices['Desktop Safari'] } }
  ],
   webServer:  {  command:  'npm  start',  port:  3000,  timeout:  120000,
reuseExistingServer: !process.env.CI },
  reporter: [['html', { open: 'never' }]]
});
```

tests/e2e.spec.ts

What it does: This file contains the actual E2E test scripts.

Test 1 (nav: Home > Volunteers -> Form): This script simulates a user navigating through the site. It goes to the homepage (/) , clicks the "volunteers" link , and then clicks the "become a volunteer" link . It uses await expect(page).toHaveURL(...) to assert that the URL changes correctly at each step.

Test 2 (form: invalid -> valid -> success): This script tests the same form logic as the Jest test, but from a real user's perspective in a browser. It fills the "Name", "Email", and "Your Skills" fields . It checks that the submit button is toBeDisabled() at first, then toBeEnabled() after valid data is entered, and finally clicks it to verify the "thank you!" message appears.

```
import { test, expect } from '@playwright/test';

test('nav: Home -> Volunteers -> Form', async ({ page }) => {
  await page.goto('/');
   await expect(page.getByRole('heading', { name: /welcome to voluntr/i
})).toBeVisible();

  await page.getByRole('link', { name: /volunteers/i }).click();
  await expect(page).toHaveURL(/\/volunteer-list$/);

     await   page.getByRole('link',   {   name:   /become   a   volunteer/i
}).click();
  await expect(page).toHaveURL(/\/volunteer-form$/);
});

test('form: invalid -> valid -> success', async ({ page }) => {
  await page.goto('/volunteer-form');
  const submit = page.getByRole('button', { name: /submit/i });
  await expect(submit).toBeDisabled();

  await page.getByLabel('Name').fill('Ada');
  await page.getByLabel('Email').fill('ada@example.com');
```

```
  await page.getByLabel('Your Skills')
    .fill('This text has more than twenty characters.');
  await expect(submit).toBeEnabled();

  await submit.click();
  await expect(page.getByText(/thank you!/i)).toBeVisible();
});
```

Performance, Accessibility & SEO Testing (Lighthouse CI)

This configuration file automates Lighthouse scans to check your site's quality scores.

.lighthouserc.json

What it does: This is the configuration file for Lighthouse CI.

Explanation:

collect: This section tells Lighthouse how to find your server (startServerCommand) and which URLs to test ("http://localhost:4173/", .../volunteer-list, .../volunteer-form) .

assert: This is the most important part. It sets the minimum score requirements for your tests to pass. Your config sets a "warning" if the scores drop below a certain level, such as minScore: 0.85 for performance and minScore: 0.90 for accessibility, best-practices, and SEO . This is how you automated the quality check.

```
{
  "ci": {
    "collect": {
      "startServerCommand": "npx serve -s build -l 4173",
                                                    "startServerReadyPattern":
"Local:.*4173|http://localhost:4173|Accepting      connections|listening
on.*4173",
      "startServerReadyTimeout": 60000,
      "settings": {
        "preset": "desktop",
        "throttlingMethod": "provided",
        "blockedUrlPatterns": ["*googletagmanager.com*"]
      },
      "url": [
        "http://localhost:4173/",
        "http://localhost:4173/volunteer-list",
        "http://localhost:4173/volunteer-form"
```

```
      ]
    },
    "assert": {
      "assertions": {
        "categories:performance": ["warn", { "minScore": 0.85 }],
        "categories:accessibility": ["warn", { "minScore": 0.90 }],
        "categories:best-practices": ["warn", { "minScore": 0.90 }],
        "categories:seo": ["warn", { "minScore": 0.90 }]
      }
    }
  }
}
```

3. Explore open source software testing tools for testing web/mobile applications.

Section	What I did?	Key files / configs	Commands	Outcome
Tools (Design + Setup)	Chosen tools: Jest/RTL, jest-axe, Playwright, Lighthouse CI. Wired up configs + npm scripts.	package.json (scripts & deps)	—	Tooling configured.

Added scripts for testing

package.json

What it does: It's the central control file for your project.

Explanation:

"scripts": This section defines the custom commands you used.
- **"test": "react-scripts test":** Runs the Jest/RTL tests.
- **"e2e": "playwright test":** Runs the Playwright E2E tests.
- **"lighthouse": "lhci autorun":** Runs the Lighthouse CI checks based on your .lighthouserc.json file.

"devDependencies": This section lists the testing tools you installed, proving you set up the environment. Key tools listed are @playwright/test , @lhci/cli , and jest-axe.

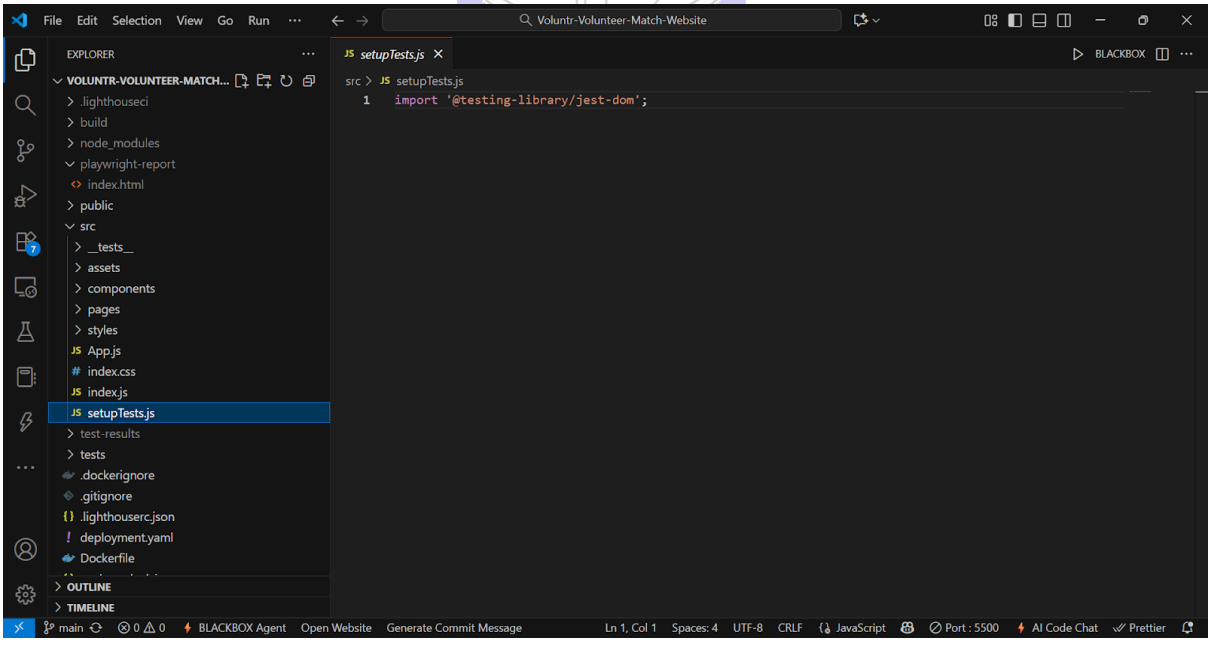
```
{
  "name": "my-app",
```

```
"version": "0.1.0",
"private": true,
"dependencies": {
  "@testing-library/dom": "10.4.0",
  "@testing-library/jest-dom": "^5.17.0",
  "@testing-library/react": "14.2.2",
  "@testing-library/user-event": "^13.5.0",
  "aria-query": "5.3.0",
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "react-icons": "^5.5.0",
  "react-router-dom": "^7.8.0",
  "react-scripts": "5.0.1",
  "web-vitals": "^2.1.4"
},

"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject",
  "e2e": "playwright test",
  "e2e:report": "playwright show-report",
  "start-static": "serve -s build -l 3000",
  "lighthouse": "lhci autorun"
},

"eslintConfig": {
  "extends": [
    "react-app",
    "react-app/jest"
  ]
},

"browserslist": {
  "production": [
    ">0.2%",
    "not dead",
    "not op_mini all"
  ],

  "development": [
```

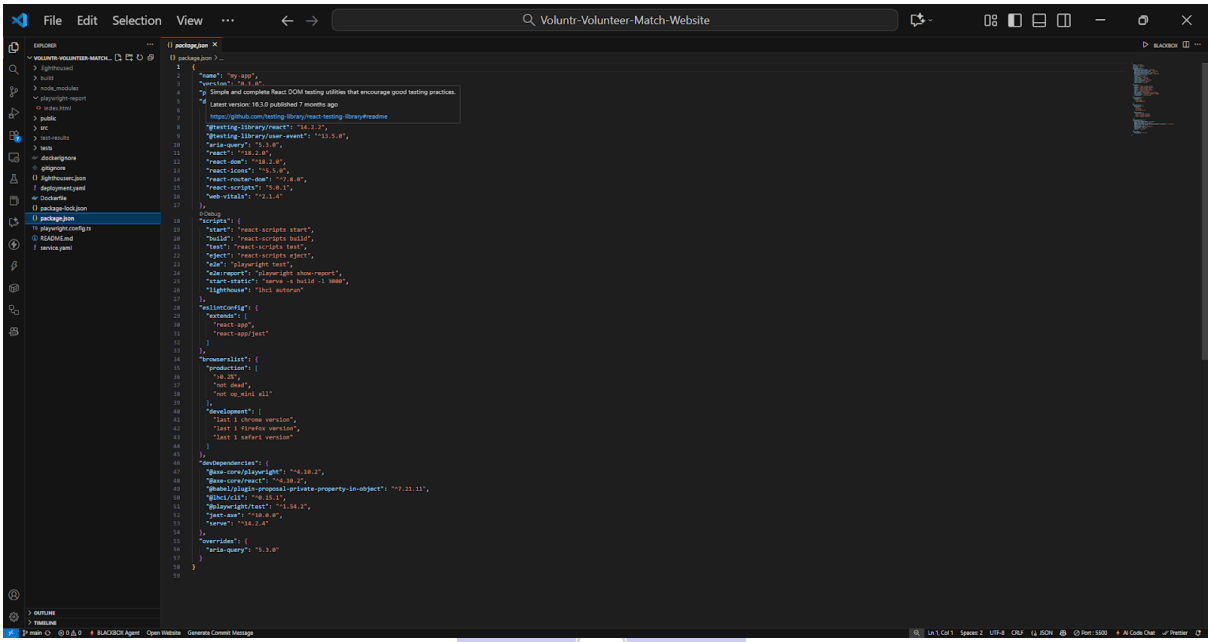
```
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  },

  "devDependencies": {
    "@axe-core/playwright": "^4.10.2",
    "@axe-core/react": "^4.10.2",
    "@babel/plugin-proposal-private-property-in-object": "^7.21.11",
    "@lhci/cli": "^0.15.1",
    "@playwright/test": "^1.54.2",
    "jest-axe": "^10.0.0",
    "serve": "^14.2.4"
  },

  "overrides": {
    "aria-query": "5.3.0"
  }
}
```

4. Document the functionality of testing tools with its features.

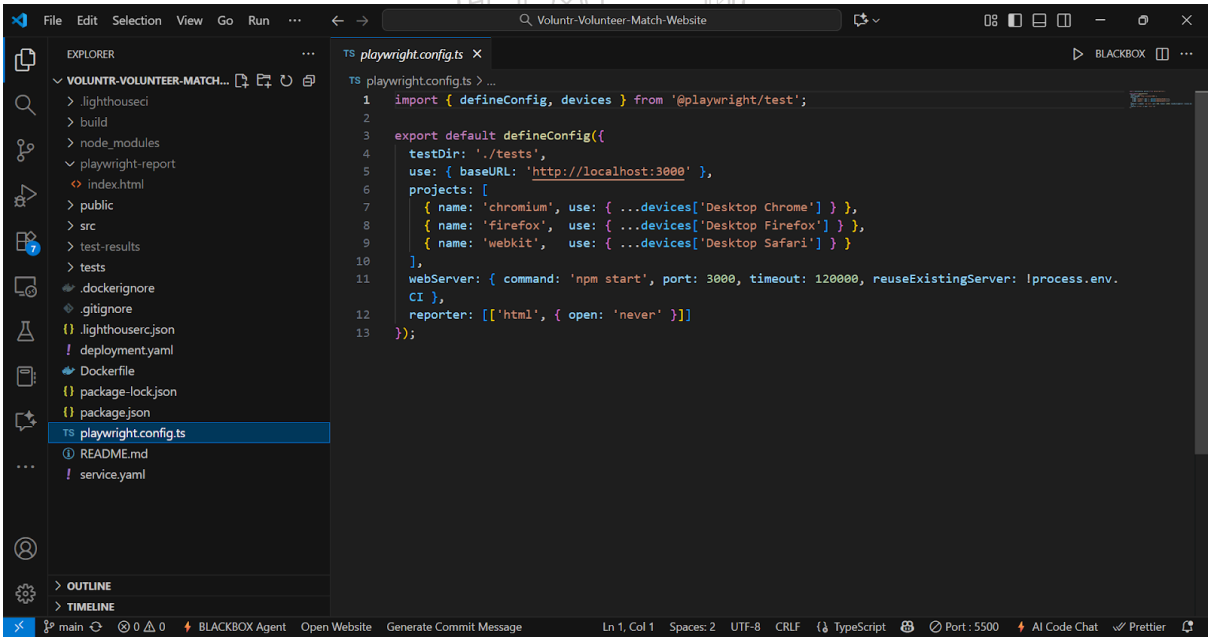
src/setupTests.js



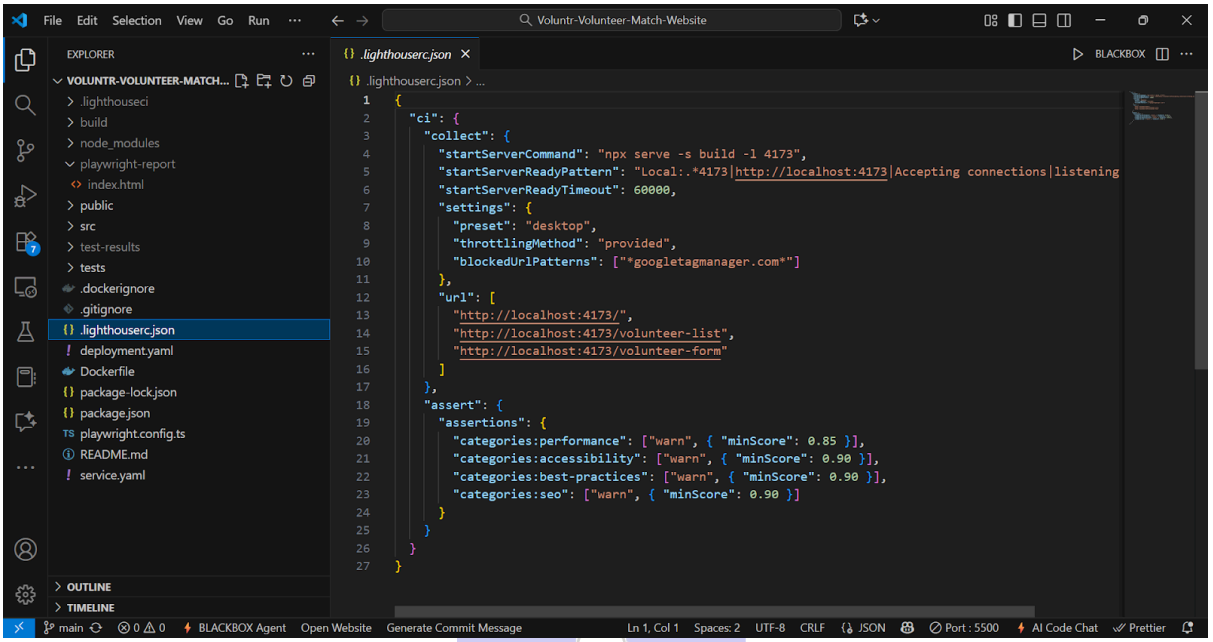
package.json



playwright.config.ts



.lighthouserc.json



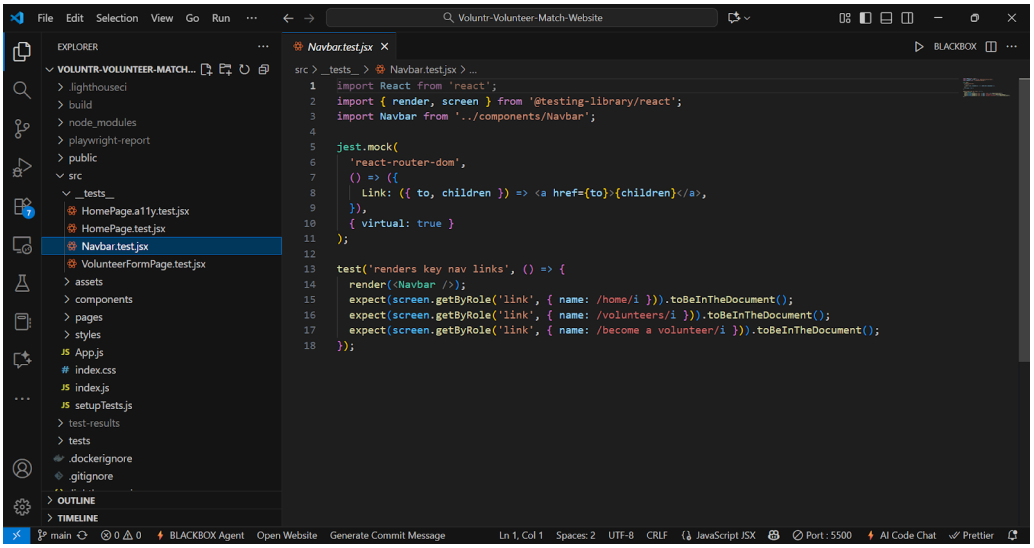
5. Document sample application features.

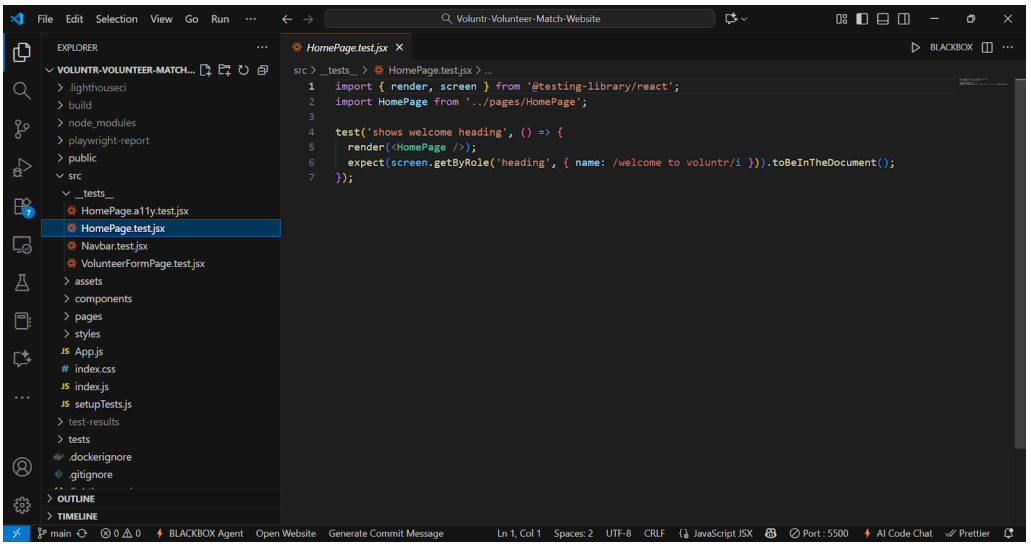
Section	What I did?	Key files / configs	Commands	Outcome
Sample Application (Build & Features)	Built Voluntr, a web application (React/CRA) with the routes Home, Volunteer List, Volunteer Form; Navbar navigation; form validation + success message.	src/pages/{HomePage, VolunteerListPage, VolunteerFormPage}.jsx, src/components/Navbar.jsx	• npm start (dev) • npm run build (prod)	App runs; build successful.

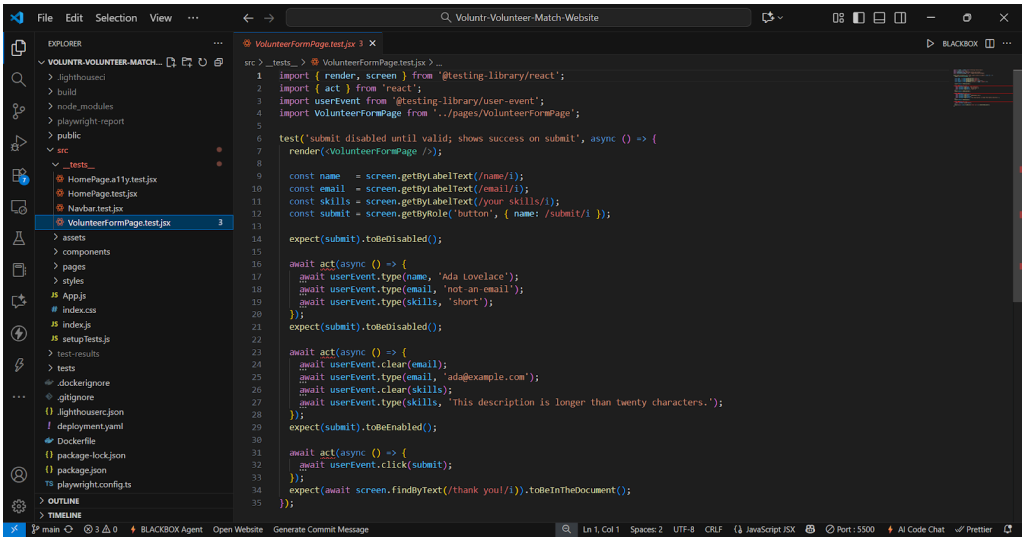
6. Create test cases and test scripts.

Unit/UI (Jest/RTL)

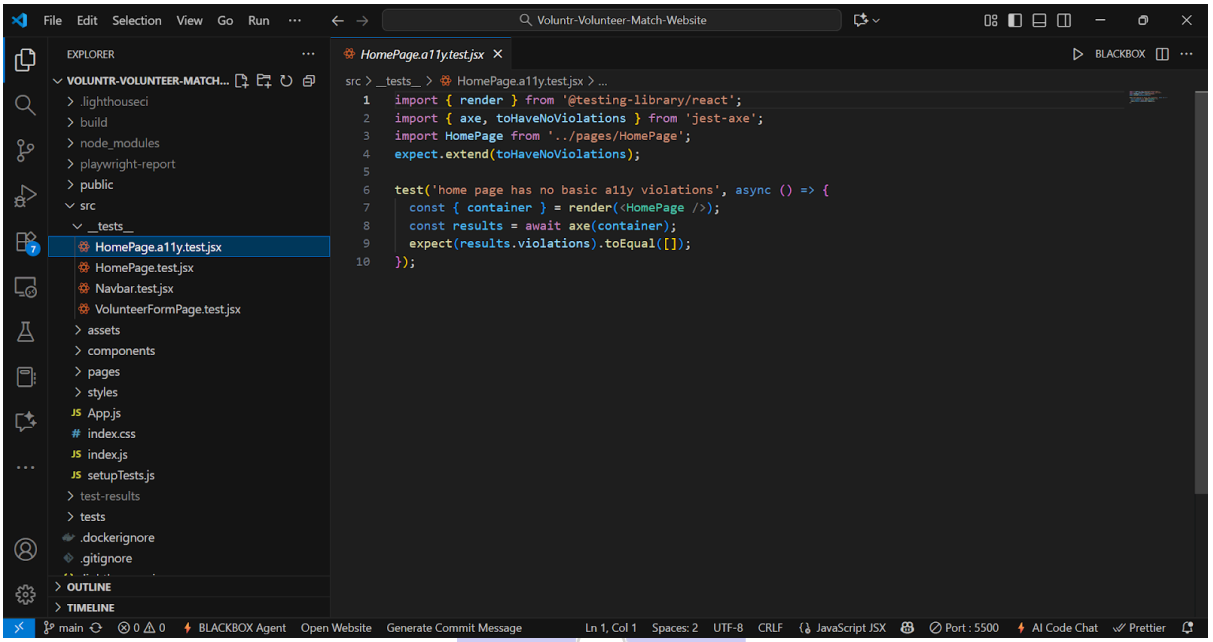
Navbar.test.jsx, HomePage.test.jsx, and VolunteerFormPage.test.jsx



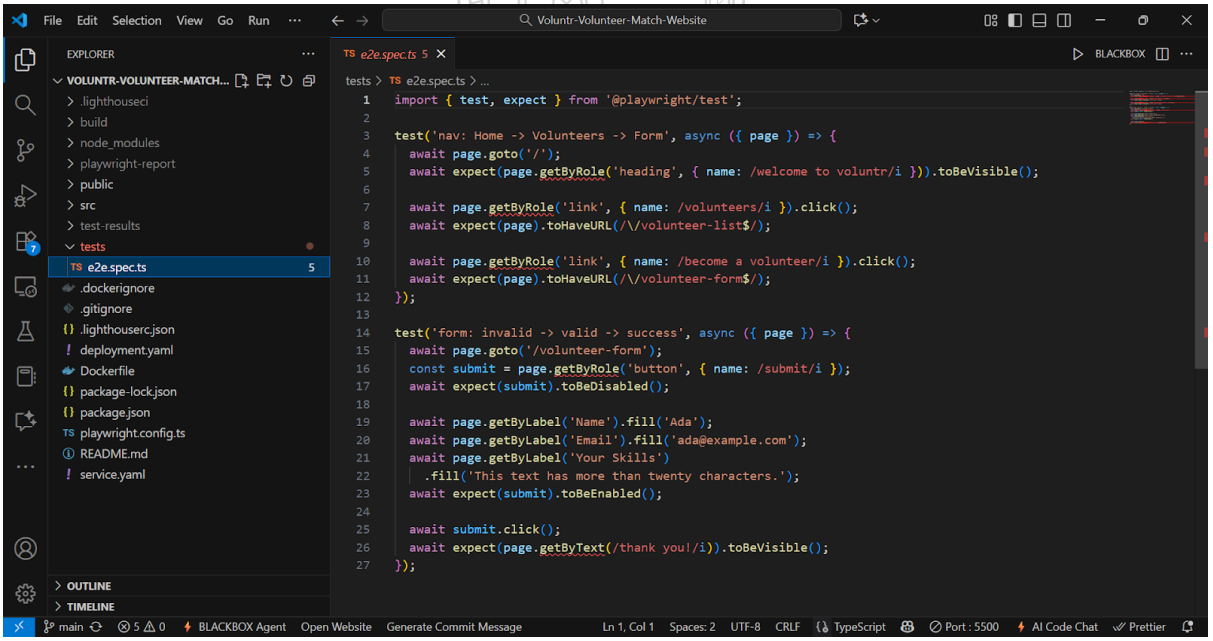




Accessibility (jest-axe): HomePage.ally.test.jsx



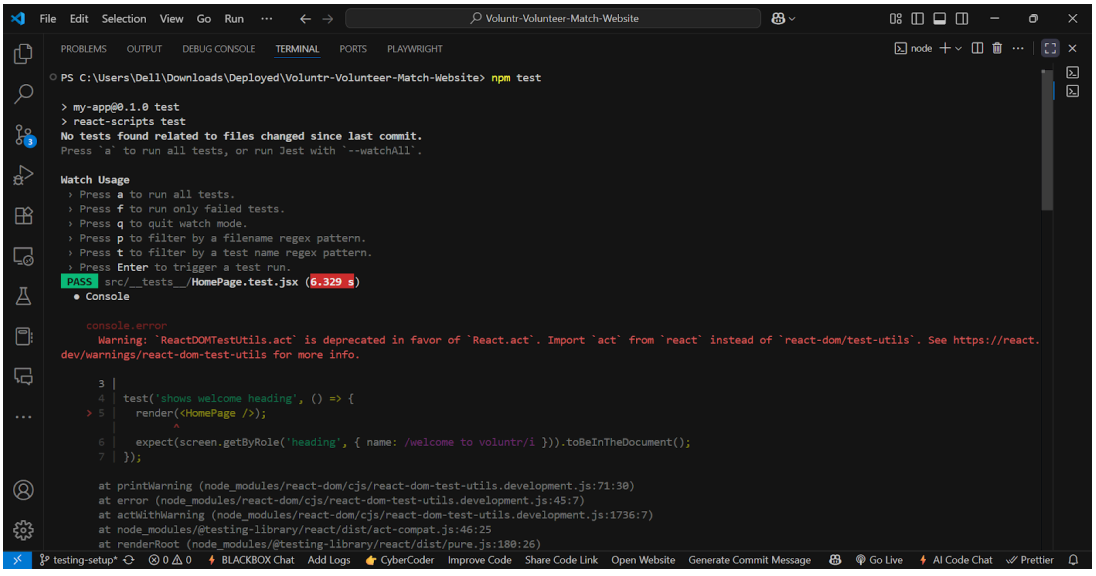
End-to-End (Playwright): tests/e2e.spec.ts

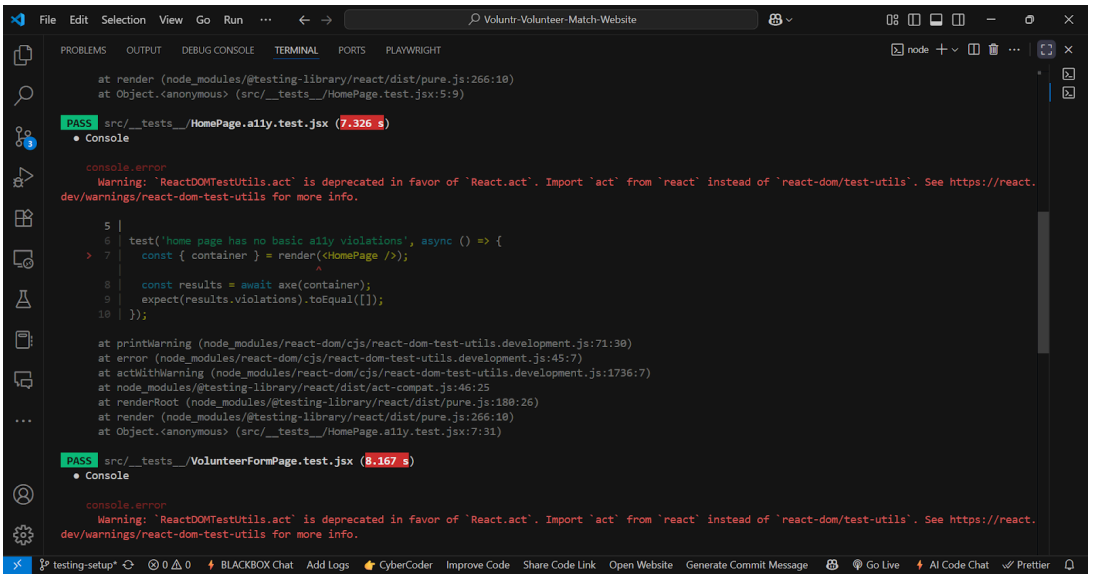


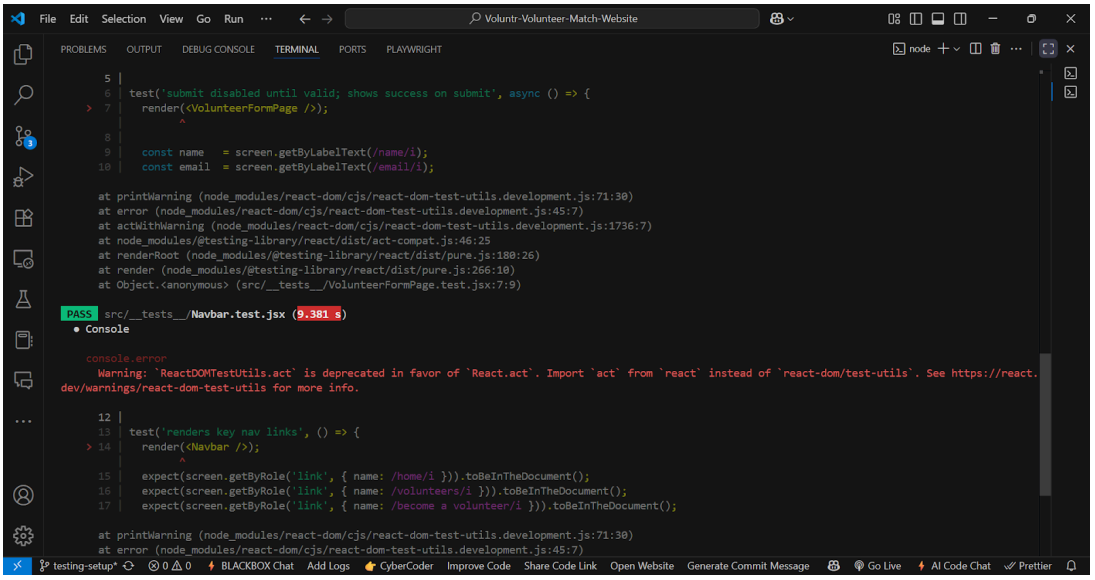
Section	What we did?	Key files / configs	Commands	Outcome
Test Execution & Results	Unit/UI: Home heading, Form validation/submit, Navbar links → PASS. Accessibility: Home with jest-axe → 0 violations. E2E (Playwright): Nav flow + form flow → PASS (HTML report generated). Lighthouse CI: ran on /, /volunteer-list, /volunteer-form via static build; perf shows warnings (≈0.54–0.74), other categories OK per config.	src/__tests__/*.test.jsx, tests/e2e.spec.ts, LHCI reports	• npm test • npm run e2e • npm run e2e:report • npm run lighthouse	Tests passed; Lighthouse executed (perf warnings noted).

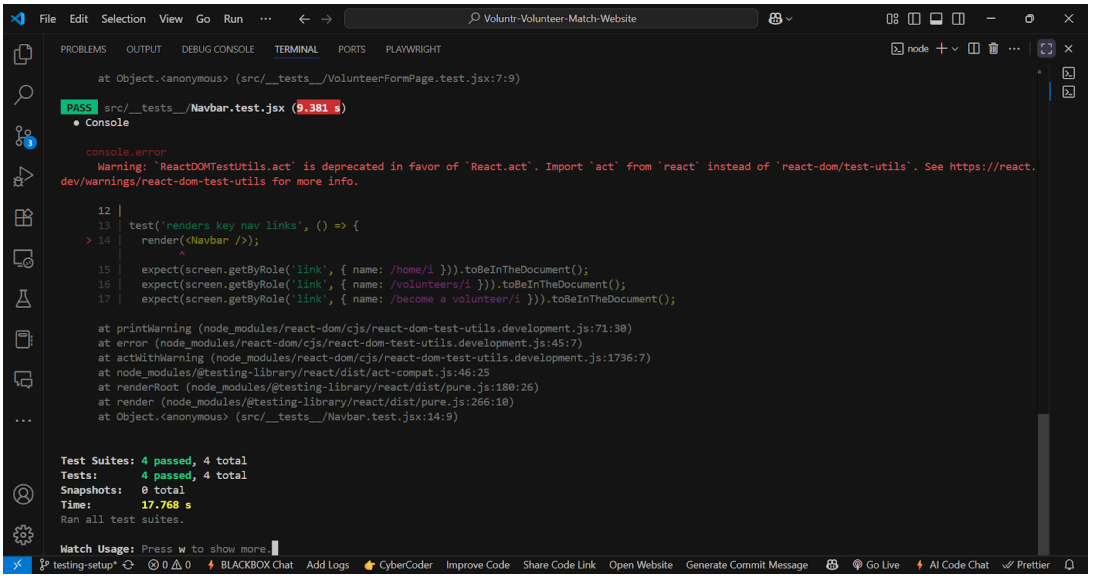
7. Run and document test cases.

Jest: The npm test command was run, resulting in "4 passed, 4 total" tests.

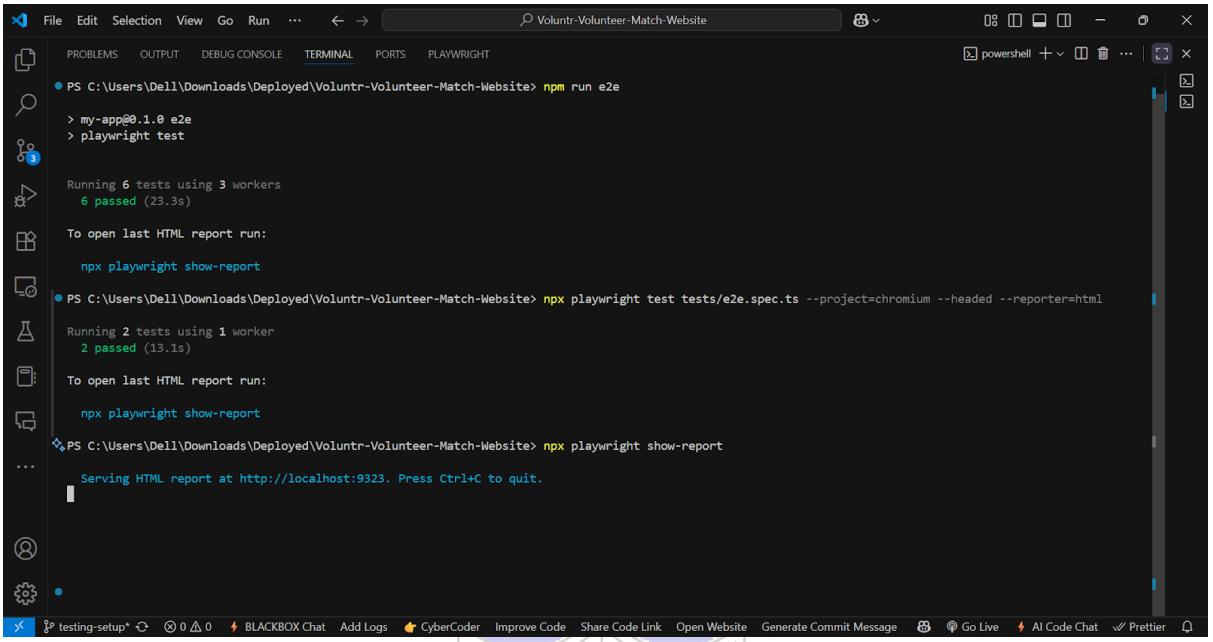



File Edit Selection View Go Run
Voluntr-Volunteer-Match-Website
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT
node
at render (node_modules/@testing-library/react/dist/pure.js:266:10)
at Object.<anonymous> (src/__tests__/HomePage.test.jsx:5:9)
PASS src/__tests__/HomePage.a11y.test.jsx (7.326 s)
Console
console.error
Warning: `ReactDOMTestUtils.act` is deprecated in favor of `React.act`. Import `act` from `react` instead of `react-dom/test-utils`. See https://react.dev/warnings/react-dom-test-utils for more info.
5 |
6 | test('home page has no basic a11y violations', async () => {
> 7 | const { container } = render(<HomePage />);
| ^
8 | const results = await axe(container);
9 | expect(results.violations).toEqual([]);
10 | });
at printWarning (node_modules/react-dom/cjs/react-dom-test-utils.development.js:71:30)
at error (node_modules/react-dom/cjs/react-dom-test-utils.development.js:45:7)
at actWithWarning (node_modules/react-dom/cjs/react-dom-test-utils.development.js:1736:7)
at node_modules/@testing-library/react/dist/act-compat.js:46:25
at renderRoot (node_modules/@testing-library/react/dist/pure.js:180:26)
at render (node_modules/@testing-library/react/dist/pure.js:266:10)
at Object.<anonymous> (src/__tests__/HomePage.a11y.test.jsx:7:31)
PASS src/__tests__/VolunteerFormPage.test.jsx (8.167 s)
Console
console.error
Warning: `ReactDOMTestUtils.act` is deprecated in favor of `React.act`. Import `act` from `react` instead of `react-dom/test-utils`. See https://react.dev/warnings/react-dom-test-utils for more info.
testing-setup* 0 0 BLACKBOX Chat Add Logs CyberCoder Improve Code Share Code Link Open Website Generate Commit Message Go Live AI Code Chat Prettier

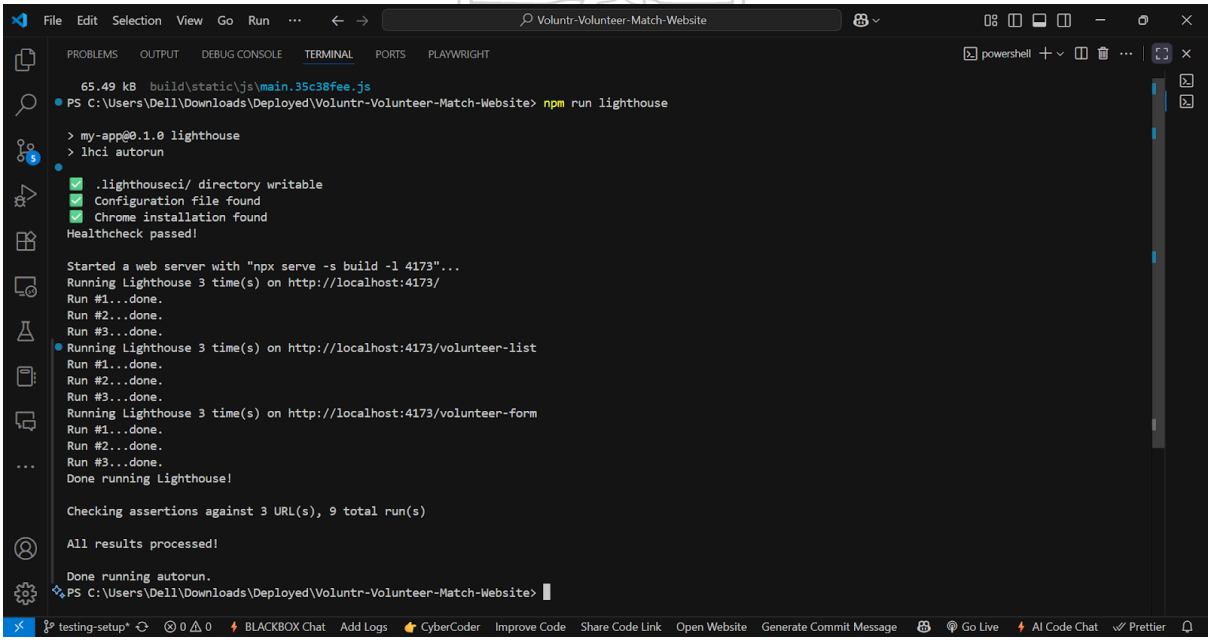

File Edit Selection View Go Run
Voluntr-Volunteer-Match-Website
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT
node
5 |
6 | test('submit disabled until valid; shows success on submit', async () => {
> 7 | render(<VolunteerFormPage />);
| ^
8 |
9 | const name = screen.getByLabelText(/name/i);
10 | const email = screen.getByLabelText(/email/i);
at printWarning (node_modules/react-dom/cjs/react-dom-test-utils.development.js:71:30)
at error (node_modules/react-dom/cjs/react-dom-test-utils.development.js:45:7)
at actWithWarning (node_modules/react-dom/cjs/react-dom-test-utils.development.js:1736:7)
at node_modules/@testing-library/react/dist/act-compat.js:46:25
at renderRoot (node_modules/@testing-library/react/dist/pure.js:180:26)
at render (node_modules/@testing-library/react/dist/pure.js:266:10)
at Object.<anonymous> (src/__tests__/VolunteerFormPage.test.jsx:7:9)
PASS src/__tests__/Navbar.test.jsx (9.381 s)
Console
console.error
Warning: `ReactDOMTestUtils.act` is deprecated in favor of `React.act`. Import `act` from `react` instead of `react-dom/test-utils`. See https://react.dev/warnings/react-dom-test-utils for more info.
12 |
13 | test('renders key nav links', () => {
> 14 | render(<Navbar />);
| ^
15 | expect(screen.getByRole('link', { name: /home/i })).toBeInTheDocument();
16 | expect(screen.getByRole('link', { name: /volunteers/i })).toBeInTheDocument();
17 | expect(screen.getByRole('link', { name: /become a volunteer/i })).toBeInTheDocument();
at printWarning (node_modules/react-dom/cjs/react-dom-test-utils.development.js:71:30)
at error (node_modules/react-dom/cjs/react-dom-test-utils.development.js:45:7)
testing-setup* 0 0 BLACKBOX Chat Add Logs CyberCoder Improve Code Share Code Link Open Website Generate Commit Message Go Live AI Code Chat Prettier


File Edit Selection View Go Run
Voluntr-Volunteer-Match-Website
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT
node
at Object.<anonymous> (src/__tests__/VolunteerFormPage.test.jsx:7:9)
PASS src/__tests__/Navbar.test.jsx (9.381 s)
Console
console.error
Warning: `ReactDOMTestUtils.act` is deprecated in favor of `React.act`. Import `act` from `react` instead of `react-dom/test-utils`. See https://react.dev/warnings/react-dom-test-utils for more info.
12 |
13 | test('renders key nav links', () => {
> 14 | render(<Navbar />);
| ^
15 | expect(screen.getByRole('link', { name: /home/i })).toBeInTheDocument();
16 | expect(screen.getByRole('link', { name: /volunteers/i })).toBeInTheDocument();
17 | expect(screen.getByRole('link', { name: /become a volunteer/i })).toBeInTheDocument();
at printWarning (node_modules/react-dom/cjs/react-dom-test-utils.development.js:71:30)
at error (node_modules/react-dom/cjs/react-dom-test-utils.development.js:45:7)
at actWithWarning (node_modules/react-dom/cjs/react-dom-test-utils.development.js:1736:7)
at node_modules/@testing-library/react/dist/act-compat.js:46:25
at renderRoot (node_modules/@testing-library/react/dist/pure.js:180:26)
at render (node_modules/@testing-library/react/dist/pure.js:266:10)
at Object.<anonymous> (src/__tests__/Navbar.test.jsx:14:9)
Test Suites: 4 passed, 4 total
Tests: 4 passed, 4 total
Snapshots: 0 total
Time: 17.768 s
Ran all test suites.
Watch Usage: Press w to show more.
testing-setup* 0 0 BLACKBOX Chat Add Logs CyberCoder Improve Code Share Code Link Open Website Generate Commit Message Go Live AI Code Chat Prettier

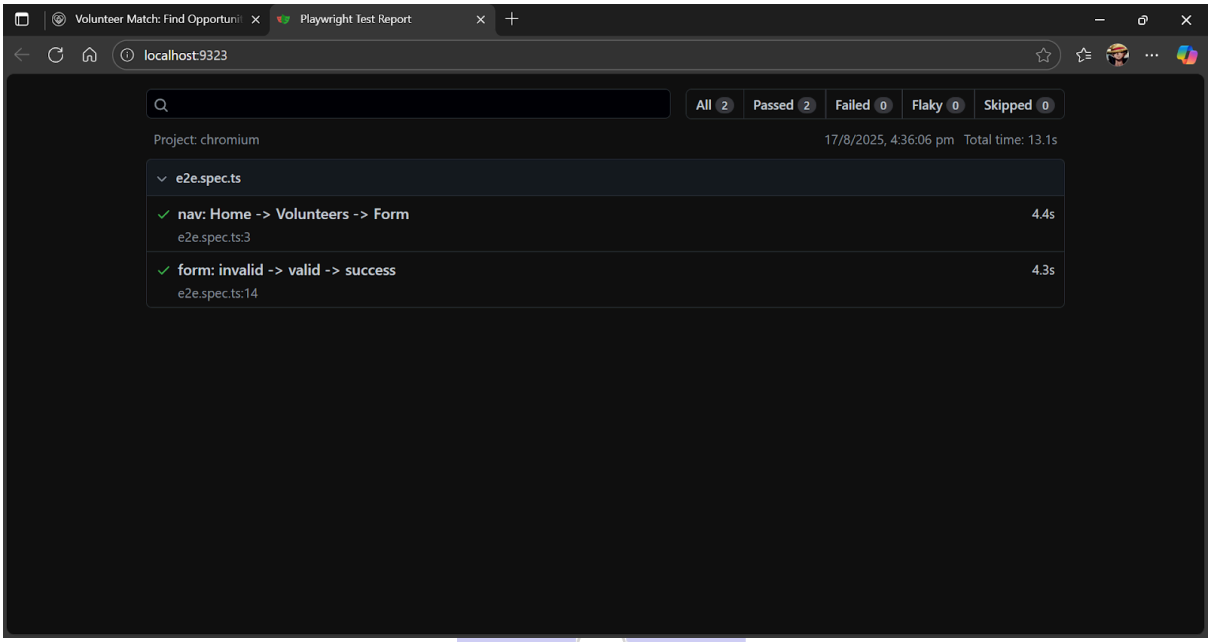
Playwright: The npm run e2e and playwright test commands were run, resulting in "passed" tests .



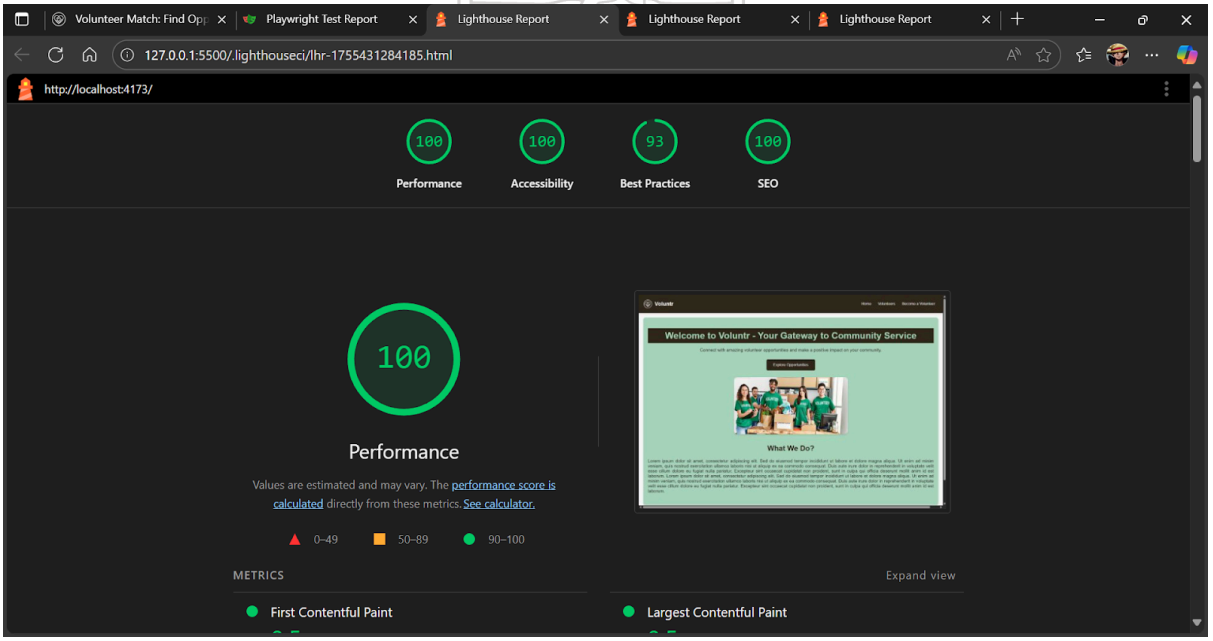
Lighthouse: The npm run lighthouse command was run, showing Lighthouse successfully executing on all 3 URLs .

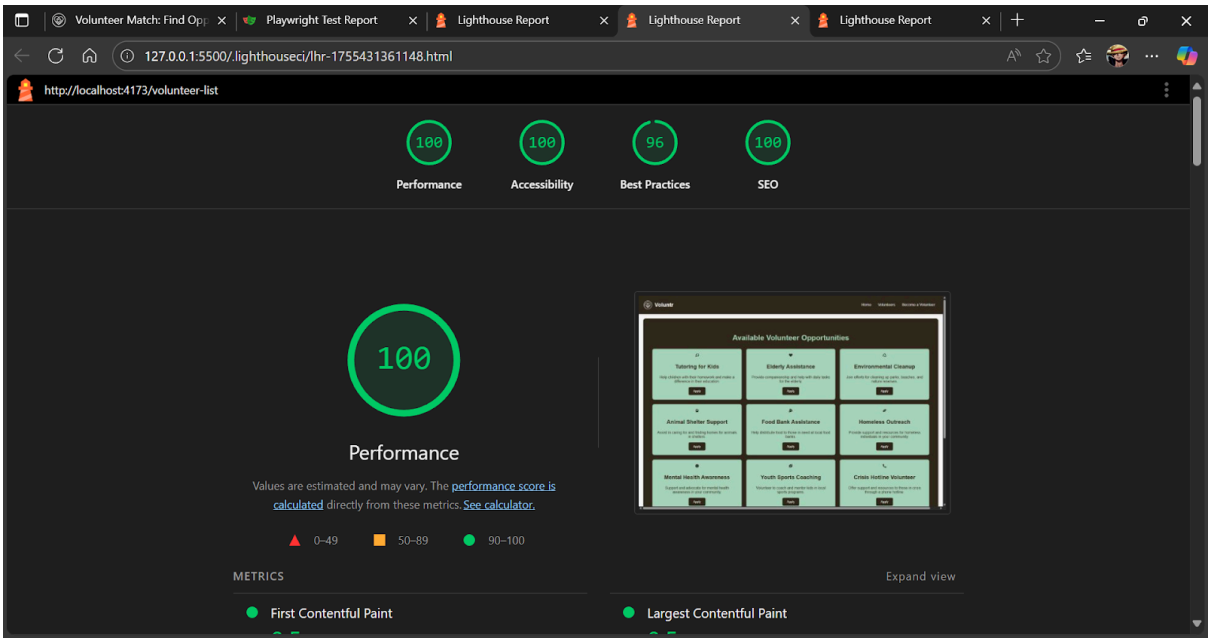


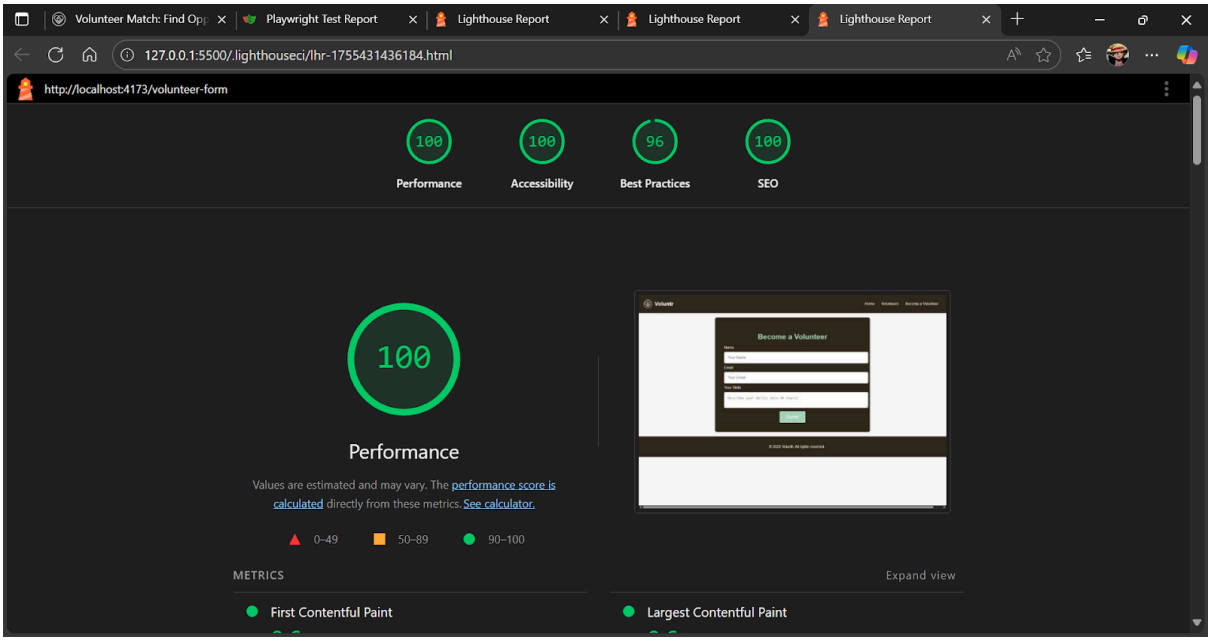
Playwright: The HTML report shows 2 "Passed" test specs .



Lighthouse: The individual reports for all three pages (/, /volunteer-list, /volunteer-form) are included, documenting the scores for Performance, Accessibility, Best Practices, and SEO .







Test Case ID	Test Case Aim	Test Case Input	Expected Output	Status
Unit / UI Testing (Jest/RTL)				
TC-01	Verify all main navigation links render correctly.	Render the <Navbar /> component.	Links containing "home", "volunteers", and "become a volunteer" are present in the document.	Pass

TC-02	Verify the main welcome heading appears on the home page.	Render the <HomePage /> component.	A heading with the text "welcome to voluntr" is present in the document.	Pass
TC-03	Verify the form's validation logic and successful submission.	1. Render <VolunteerFormPage />. 2. Enter invalid data (e.g., short skill description). 3. Enter valid data (e.g., valid email, long skill description). 4. Click the "submit" button.	1. Submit button is disabled. 2. Submit button remains disabled. 3. Submit button becomes enabled. 4. Text "thank you!" appears on the screen.	Pass
Accessibility Testing (jest-axe)				
TC-04	Verify the home page has no basic accessibility violations.	Render the <HomePage /> component and scan with axe.	The axe tool returns zero violations (violations.toEqual([])).	Pass
End-to-End Testing (Playwright)				
TC-05	Verify the main user navigation flow from Home to the Form.	1. Go to Home page (/).2. Click "volunteers" link.3. Click "become a volunteer" link.	1. Home page heading is visible. 2. URL changes to /volunteer-list. 3. URL changes to /volunteer-form.	Pass
TC-06	Verify the form's full end-to-end validation and submission flow.	1. Go to /volunteer-form. 2. Fill all fields with valid data. 3. Click "submit".	1. Submit button is initially disabled. 2. Submit button becomes enabled. 3. "thank you!" text is visible.	Pass
Performance & SEO Testing (Lighthouse CI)				
TC-07	Audit Performance, Accessibility, Best Practices, and SEO for the	Run Lighthouse CI on the Home page (/)	Scores meet or exceed the thresholds defined in .lighthouserc.json (e.g., Perf ≥ 0.85,	Pass(Scores: 99, 100, 100, 100)

	Home page.		A11y ≥ 0.90).	
TC-08	Audit Performance, Accessibility, Best Practices, and SEO for the Volunteer List page.	Run Lighthouse CI on the Volunteer List page (/volunteer-list).	Scores meet or exceed the thresholds defined in .lighthouserc.json.	Pass(Scores: 100, 100, 96, 100)
TC-09	Audit Performance, Accessibility, Best Practices, and SEO for the Volunteer Form page.	Run Lighthouse CI on the Volunteer Form page (/volunteer-form) .	Scores meet or exceed the thresholds defined in .lighthouserc.json.	Pass(Scores: 99, 100, 96, 100)

Questions:

1. **Is automation always advantageous? When should one decide to automate test cases?**

   Ans: Automation is not always advantageous, and its use should be decided based on specific factors related to the project. Some key considerations include:

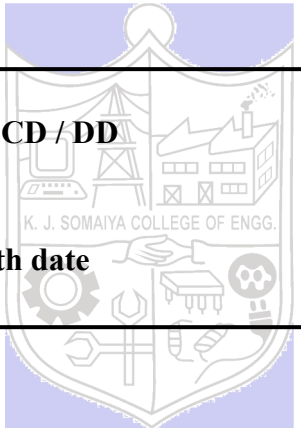
   1) **Repetitiveness:** Test automation is ideal for repetitive tasks, such as regression testing, where the same test cases are run frequently. If a test is going to be run repeatedly in different cycles, automating it can save time and effort in the long term.
   2) **Complexity of Tests:** When test scenarios are complex or difficult to perform manually, automation can help in executing and managing those tests more effectively. For instance, automated tests can simulate user interactions over extended periods or on different devices, which would be hard to replicate manually.
   3) **Project Size:** For larger projects with long lifecycles, test automation ensures that existing features continue to work as the application evolves, preventing regression issues. However, if the project is small, the overhead of setting up test automation might not be worth it.
   4) **Frequency of Changes:** If the software undergoes frequent changes or bug fixes, manual testing may become inefficient and error-prone. In this case, automation helps ensure that previously tested functionalities continue to work after new updates.
   5) **Cost and Time:** While setting up test automation might require an initial investment in terms of time and resources, it pays off if there are long-term

testing cycles or high-frequency testing requirements. Automation might not be justified in small, one-time projects due to the upfront effort involved.

Outcomes: CO1 – Understand software testing concepts and strategies.

Conclusion: (Conclusion to be based on outcomes)

In conclusion, test automation is a crucial component of modern software testing, enhancing the speed, accuracy, and reliability of tests, particularly for repetitive tasks like regression testing. By automating these tests, software products can be maintained efficiently and continuously verified, ensuring stability even after updates. The use of open-source software testing tools provides cost-effective solutions, allowing developers to test both web and mobile applications comprehensively. Test automation, when strategically implemented, can significantly improve the software development lifecycle by reducing manual testing effort and identifying defects early. Ultimately, it ensures the long-term quality and robustness of the application.



Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. Software Testing Principles and Practices, Naresh Chauhan, Second Edition, Oxford Higher Education
2. Practical Software Testing, Ilene Burnstein, First Edition, Springer International Edition
3. Effective Methods for Software Testing, Third edition by Willam E. Perry, Wiley Publication
4. http://www.softwaretestinghelp.com/most-popular-web-application-testing-tools/
5. http://www.softwaretestinghelp.com/best-mobile-testing-tools/
6. http://www.seleniumhq.org/
7. http://appium.io/
